

Real-world (make it feel alive with streaming) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Simple Version

```
from openai import OpenAI

client = OpenAI()

stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Tell me a short story about a robot."}]
    stream=True,
)

for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")

print()
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version — 2 approaches

Approach 1 — print-as-you-go, no memory of the full text

```
from openai import OpenAI

client = OpenAI()

def stream_message(history: list[dict]) -> None:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
    print()
```

This is the minimum: it makes the reply feel instant, but it throws each piece away as soon as it's printed — you can never use the full reply afterward (e.g. to save it into `history`).

Approach 2 — print-as-you-go, AND keep the full text (what you'd actually ship)

```
from openai import OpenAI

client = OpenAI()

def stream_message(history: list[dict]) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )
    full_reply = ""
    for chunk in stream:
```

```
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")
        full_reply += piece
print()
return full_reply

history = [{"role": "system", "content": "You are a helpful assistant."}]
history.append({"role": "user", "content": "Tell me a short story about a robot."})
reply = stream_message(history)
history.append({"role": "assistant", "content": reply})
```

Why Approach 2 is what you'd actually ship: a chat feature needs both things at once — the user sees the reply appear instantly, AND your code needs the completed text afterward to keep the conversation's memory going (exactly what the Intermediate exercise built). Approach 1 looks simpler but is a dead end the moment you need multi-turn memory, which almost every real chat feature does. Building the accumulator in from the start costs one extra line and saves you from a rewrite later.